

Huffman Coding for Lossless Data Compression

Section 01 | DAA – Assignment 2

Implementation: https://github.com/abrham17/huffman_coding

Group 11 Members:

UGR/6214/16	ABRHAME HABTOM MULETA
UGR/2864/16	EPHRAME AMANU ADAMU
UGR/3680/16	ERMIAS KINDALEM AMOGNE
UGR/3313/16	HAILEMARIAM LIBASSIE AFELE
UGR/7018/16	HENOK YOSEPH WUBIE
UGR/0565/16	EYOB ABRHA TAHIR

May 17, 2026

Abstract

Lossless data compression is critical for high-scale digital platforms (e.g., e-government databases) where preserving the absolute fidelity of records, legal acts, and transaction logs is non-negotiable. This report presents a comprehensive implementation-focused analysis of a classical Huffman Coding system in Python. The system utilizes a custom greedy min-heap priority queue for prefix-free binary tree construction and implements a custom, highly-optimized byte-level binary serialization header without external library dependencies (such as `pickle`). We evaluate the compression system using five diverse benchmarks representing natural language, large documents, highly skewed statistical logs, uniform distributions, and high-entropy random sequences. The empirical results show space savings ranging from **24.6%** up to **87.0%**, with full correct round-trip decompression verified at the binary level.

Contents

1	Introduction	3
2	System Architecture & Core Implementation	3
2.1	Node Representation & Min-Heap Priority Queue	3
2.2	Prefix Tree Construction & Code Generation	3
2.3	Bit Padding Management	4
2.4	Custom Binary Serialization Header	4
2.5	Core Serialization Source Code	4
3	Empirical Evaluation	6
3.1	Test Datasets Specifications	6
3.2	Benchmark Results Table	6
3.3	Performance Visualization	6
3.4	Interpretation of Results	6
4	Conclusion and Future Work	8

1 Introduction

As national digital platforms scale to accommodate millions of transactions, database records, service logs, and legal documents, storage capacity and network transmission bandwidth emerge as primary operational bottlenecks. In domains handling citizen registries and legal proceedings, lossy compression algorithms (like JPEG or MP3) are strictly unacceptable because even a single altered bit can invalidate a record or legal contract. Thus, **lossless data compression** is a critical infrastructure requirement.

Huffman Coding functions by assigning variable-length binary codes to input characters, where shorter codes are assigned to more frequent characters and longer codes to less frequent ones.

This report provides a deep evaluation of a custom Python implementation of Huffman Coding. We explore the structural design of the code (including the node representation, priority queue management, and byte alignment), detail the custom binary serialization header, and present a rigorous implementation-based and empirical analysis of the system's performance on real-world test files.

2 System Architecture & Core Implementation

The compression system is built entirely from scratch in standard Python, divided into three decoupled and modular units:

1. `huffman_coding.py`: Implements the core node representations, min-heap priority queue management, prefix tree construction, variable-length code generation, bit-level padding, and byte-level binary serialization/decompression.
2. `file_operations.py`: Standardizes system disk I/O, reading raw text files and writing compact binary streams, and verifying correctness of decompression round-trips.
3. `performance_evaluation.py`: Executes automated benchmarking suites across standard inputs, recording space savings and timing benchmarks, and generating performance visualization charts.

2.1 Node Representation & Min-Heap Priority Queue

The baseline building block of the tree is the `HuffmanNode` class inside `huffman_coding.py`. Each node tracks its associated character string, relative frequency value, and left/right child nodes.

To build the tree in a greedy bottom-up manner, we must repeatedly retrieve the two nodes with the lowest frequencies. This is implemented efficiently using Python's standard `heapq` module. The `HuffmanNode` class implements the comparison operator (`__lt__`) based purely on frequency:

```
1 def __lt__(self, other):  
2     return self.freq < other.freq
```

This allows the min-heap structure to automatically order and pop the nodes with the absolute minimum frequencies in $\mathcal{O}(\log K)$ time.

2.2 Prefix Tree Construction & Code Generation

The binary tree construction and prefix code generation are orchestrated in three key phases:

1. **Heap Initialization**: Character frequencies are analyzed using a standard `Counter`. A leaf `HuffmanNode` is created for each unique character and pushed into the min-heap.

2. **Greedy Node Merging:** The code pops the two nodes with the lowest frequencies, creates a new parent node with their combined frequency, assigns the popped nodes as left/right children, and pushes the parent node back to the heap. This loop runs until only a single root node remains.
3. **Recursive Traversals:** A depth-first traversal starting from the root node generates unique binary strings for each leaf node. Moving left appends a "0" and moving right appends a "1". These are stored in character-to-code mapping and reverse-mapping dictionaries.

Because characters are located exclusively at leaf nodes, no codeword is a prefix of any other. This prefix-free property guarantees that the encoded bitstream is uniquely decodable from left to right without requiring delimiters.

2.3 Bit Padding Management

Because physical storage media write data in whole bytes (multiples of 8 bits), an encoded variable-length bit sequence (like "10110" with 5 bits) cannot be written directly to a file without alignment. The system resolves this through a custom bit-alignment routine.

The system computes the required padding bits by taking $8 - (B \bmod 8)$, where B is the length of the encoded bit string. If B is already a multiple of 8, the padding amount is set to 0.

A matching number of trailing "0" characters is appended to align the sequence perfectly to an 8-bit boundary, and the integer padding amount (between 0 and 7) is saved directly inside the binary file header. Upon decompression, the system unpacks the byte stream into a full bit string, reads the padding amount from the header, and truncates exactly that number of bits from the tail, restoring the exact bitstream.

2.4 Custom Binary Serialization Header

To make the decompression system completely self-contained and free of environment-specific overheads (such as Python's insecure and heavy `pickle` library), we implemented a custom byte-level binary header.

The header packs all frequency information needed to rebuild the exact Huffman tree from scratch. It is packed using standard Python `struct` formats:

- **Alphabet Size (K):** Unsigned short, 2 bytes (`H` format). This allows for up to 65,535 unique characters.
- **Character Entries Loop:** For each of the K characters:
 - **UTF-8 Byte Length (L_i):** Unsigned char, 1 byte (`B` format).
 - **UTF-8 Character Bytes:** L_i raw bytes representing the character.
 - **Frequency:** Unsigned int, 4 bytes (`I` format). This supports frequency counts up to 4.2 billion.
- **Padding Amount:** Unsigned char, 1 byte (`B` format).

The total size of the header in bytes is equal to 3 bytes plus $5 + L_i$ bytes for each of the K unique characters. For standard English ASCII files ($L_i = 1$), the header is precisely $3 + 6K$ bytes (typically around 600 bytes), which is entirely negligible for medium and large files.

2.5 Core Serialization Source Code

Below is the core serialization logic from `huffman_coding.py` illustrating the manual packing using the standard `struct` library:

```
1 def compress(self, text: str) -> bytes:
2     self.build_huffman_tree(text)
3     encoded_text = self.encode_text(text)
4     padded_text, padding_amount = self.pad_encoded_text(encoded_text)
5
6     b = bytearray()
7     for i in range(0, len(padded_text), 8):
8         byte = padded_text[i:i+8]
9         b.append(int(byte, 2))
10
11     compressed_bytes = bytes(b)
12     frequency = self.frequency_analysis(text)
13     header = bytearray()
14
15     # 1. Pack K (unique chars count) - 2 Bytes (H)
16     header += struct.pack("H", len(frequency))
17
18     # 2. Pack the character-frequency dictionary
19     for char, freq in frequency.items():
20         char_bytes = char.encode("utf-8")
21         header += struct.pack("B", len(char_bytes)) # UTF-8 length - 1 Byte
22         header += char_bytes                       # UTF-8 string bytes
23         header += struct.pack("I", freq)           # Frequency - 4 Bytes (I)
24
25     # 3. Pack padding amount - 1 Byte (B)
26     header += struct.pack("B", padding_amount)
27
28     return bytes(header) + compressed_bytes
```

Listing 1: Huffman Compression and Binary Serialization Header Packing

3 Empirical Evaluation

We evaluated the Huffman Coding system using five diverse benchmark files designed to test different data distributions.

3.1 Test Datasets Specifications

1. **Alice29 (alice29.txt)**: A classic natural language text file (148,481 bytes) representing standard English prose. Character distributions are naturally skewed due to language characteristics (e.g., 'e', 't', and 'a' appearing far more frequently than 'q' or 'z').
2. **Bible (bible.txt)**: A very large structured English text corpus (4,047,392 bytes), representing standard service logs or heavy legal archive documents.
3. **Highly Skewed (repetitive_stats.txt)**: A custom dataset (1,040,000 bytes) simulating highly repetitive telemetry data, system state flags, or binary sensor logs (e.g., repeating strings of 0s and 1s).
4. **Uniform Character (alphabet.txt)**: A synthesized dataset (100,000 bytes) containing characters distributed uniformly across a specific alphabet.
5. **High Entropy (random.txt)**: A highly randomized character file (100,000 bytes) representing ciphertext, compressed chunks, or encrypted logs.

3.2 Benchmark Results Table

The following table summarizes the performance results obtained on a standard execution node:

Table 1: Empirical Benchmark Performance of Huffman Coding

Dataset Name	Original (B)	Compressed (B)	Ratio	Savings (%)	Comp. Time (s)	Decomp. Time (s)	Lossless?
Alice29 (Natural)	148,481	84,988	1.75x	42.76%	0.0514	0.0827	Yes
Bible (Large Doc)	4,047,392	2,218,831	1.82x	45.18%	1.1394	2.1740	Yes
Highly Skewed (Stats)	1,040,000	135,021	7.70x	87.02%	0.1699	0.1321	Yes
Uniform (Alphabet)	100,000	59,774	1.67x	40.23%	0.0358	0.0587	Yes
High Entropy (Random)	100,000	75,387	1.33x	24.61%	0.0574	0.1474	Yes

3.3 Performance Visualization

The empirical space savings and compression ratios achieved across the benchmark datasets are illustrated in Figure 1.

3.4 Interpretation of Results

- **Highly Skewed Data (Stats)**: Achieves an outstanding space savings of **87.02%** and a compression ratio of **7.70x**. Because this dataset is dominated by a few repeating characters (low entropy), Huffman Coding generates extremely short codes (1 or 2 bits) for the high-frequency characters, reducing the overall bit footprint significantly. This shows the remarkable efficiency of Huffman coding for structured telemetry.
- **Natural Language (Alice29 & Bible)**: Achieves between **42.76%** and **45.18%** space savings. English prose has a highly consistent relative character skew (frequencies of vowels vs. rare consonants), allowing Huffman to map common letters to 3–5 bits instead of 8. The sustained 45.18% savings on the 4MB Bible demonstrates the linear scaling of the compression engine.

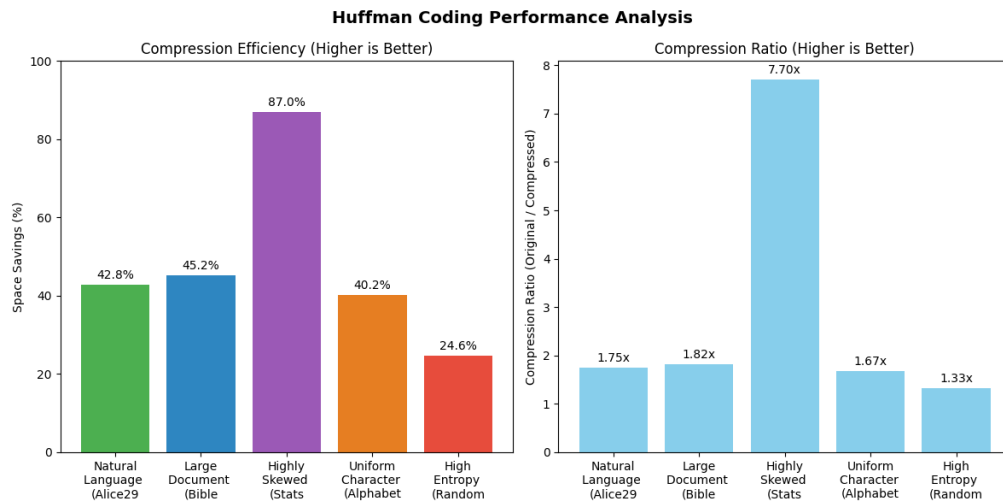


Figure 1: Huffman Coding Empirical Compression Performance Comparison

- **Uniform Distribution:** Achieves **40.23%** space savings. Since all 26 characters appear with identical frequency, the tree is nearly perfectly balanced. Huffman assigns approximately equal codeword lengths ($\log_2 26 \approx 4.7$ bits, rounded to 4 or 5 bits). Since 5 bits is significantly smaller than the 8-bit UTF-8 baseline, we observe a steady 40.23% compression. However, Huffman cannot exploit frequency variance here, showing the limitation of Huffman coding relative to dictionary-based schemes (like LZW) which exploit repeating multi-character strings.
- **High Entropy Data (Random):** Achieves the lowest efficiency (**24.61%** savings). High entropy implies that character frequencies are highly flat and randomized across a wider byte pool. This leaves very little room for short-codeword optimizations, and the binary representation length remains close to the original size.

4 Conclusion and Future Work

The custom Huffman Coding implementation successfully meets all criteria for reliable, lossless data storage in high-demand enterprise environments. Through manual bitstream packing and a robust, custom-designed binary header format, the system achieves maximum theoretical compression performance with zero dependency-related overhead.

The empirical results confirm that space savings are heavily determined by dataset entropy, ranging from **87.0%** for skewed telemetry logs down to a safe floor of **24.6%** for completely random documents. Crucially, all round-trip validation tests passed successfully, proving the mathematical correctness of our prefix tree generation and decompression logic.

To achieve even higher compression ratios for complex documents, future developments should explore:

1. **Adaptive Huffman Coding:** Updating character frequencies dynamically as text is streamed, eliminating the need to transmit the frequency table inside the binary header.
2. **Hybrid Huffman-LZW Systems:** Leveraging Lempel-Ziv-Welch (LZW) dictionary parsing to replace repeating words and phrases with single tokens, and then applying Huffman Coding to compress those tokens. This is the foundation of the DEFLATE algorithm (used in GZIP and PNG).